

# ANALISIS PERFORMA KOMUNIKASI DATA REST API DAN GRAPHQL DALAM PENYAJIAN DATA DETEKSI OBJEK MENGGUNAKAN YOLO26

*Performance Analysis of REST API and GraphQL in Delivering Object Detection Data Using YOLO26*

---

Jeihan Ilham Kusumawardhana — 221524042

Sidang Tugas Akhir · Tipe TA: Riset (Experiment tools)

Program Sarjana Terapan Teknik Informatika · Jurusan Teknik Komputer dan Informatika · Politeknik Negeri Bandung · 2026

*Pembimbing: [VERIFY: nama & NIP Pembimbing I dan II]*

# PT Ganesha Digital Solusi: satu sumber data deteksi, kebutuhan klien yang heterogen.

**Dashboard web:** membutuhkan data deteksi visual utuh.

**Aplikasi mobile / IoT:** membutuhkan efisiensi kuota — hanya ringkasan; hemat bandwidth dan baterai.

**Yang dipertaruhkan mitra:** efisiensi maintenance endpoint API dan optimalisasi bandwidth, tanpa mengorbankan stabilitas server pada high concurrency.

**Leher botolnya di penyajian:** penyajian data yang lambat menggerus manfaat inferensi real-time meski akurasi model tinggi.

# Dua paradigma penyajian: REST kaku per-endpoint, GraphQL fleksibel per-field — dan fleksibilitas itu tidak gratis.

**REST (Fielding, 2000):** gaya arsitektur berbasis resource dengan struktur respons kaku → over-fetching (data tak dibutuhkan ikut terkirim) atau under-fetching (klien memanggil banyak endpoint; pola N+1).

**GraphQL:** query language + runtime; klien menentukan field yang dibutuhkan dalam satu permintaan (declarative data fetching).

**Biaya di sisi server:** setiap field diproses resolver — jumlah field, kedalaman relasi, dan jumlah objek menentukan beban; dan default POST GraphQL tidak cacheable secara HTTP.

**Konsekuensi:** perbandingan keduanya harus dievaluasi secara empiris per skenario kebutuhan data — bukan diklaim universal.

# Kesenjangan: belum ada perbandingan yang fair pada payload deteksi objek.

“Belum ada studi yang membandingkan REST API dan GraphQL pada payload deteksi objek dengan caching, pola akses, dan bobot payload dikontrol secara eksplisit sebagai variabel bebas, menggunakan sumber data dan jalur akses data yang sama bagi kedua protokol.”

**Studi terdahulu:** memakai data relasional generik; latensi tercampur I/O basis data; caching jarang dikontrol.

**Literatur VDBMS:** fokus pada optimasi internal engine; bagaimana hasil query visual didistribusikan ke klien melalui protokol delivery web API tidak dibahas.

## Bekukan inferensi, samakan jalur data, kontrol caching — sehingga selisih yang terukur hanya berasal dari protokol.

**YOLO26 sebagai black-box:** inference engine deterministik; hasil deteksi & tracking dibekukan ke korpus relasional SQLite secara luring, sebelum eksperimen berjalan.

**Jalur data identik:** REST dan GraphQL membaca sumber data yang sama melalui shared data-access layer.

**Yang divariasikan:** caching, pola akses, entropi query, dan bobot payload sebagai variabel bebas eksperimen utama.

**Jaminan atribusi:** setiap perbedaan metrik dapat ditelusuri ke karakteristik arsitektural protokol (over-fetching REST / resolver traversal overhead GraphQL) serta interaksinya dengan caching — bukan ke perbedaan jalur akses data.

## Tiga pertanyaan penelitian (verbatim dari laporan).

RQ1. Bagaimana perbedaan kinerja dasar—response time, throughput, dan penggunaan sumber daya (CPU dan memori)—antara REST API dan GraphQL ketika keduanya menyajikan data hasil deteksi objek dari sumber data dan jalur akses data yang identik, tanpa manfaat caching?

RQ2. Bagaimana pengaruh caching, pola akses data, entropi bentuk query, bobot payload, dan konsolidasi round-trip terhadap selisih kinerja (latency, cache-hit rate, dan efisiensi transfer data) antara REST API dan GraphQL?

RQ3. Pada kombinasi kondisi apa REST API tetap lebih unggul, dan pada kondisi apa GraphQL mendekati atau menyamai REST API (titik crossover), sehingga dapat disusun matriks rekomendasi arsitektural pemilihan protokol?

# Empat hipotesis substantif; H0 per sel: tidak ada perbedaan signifikan (Mann-Whitney U).

## H1 • H2

---

H1. Pada kondisi caching nonaktif dan payload ringan, REST API memiliki latency lebih rendah daripada GraphQL, karena permintaan REST dipetakan langsung ke endpoint tanpa overhead parsing Abstract Syntax Tree (AST) dan resolver traversal. (menjawab RQ1)

H2. Pada kondisi caching aktif dengan pola akses berulang (zipfian), selisih latency antara REST API dan GraphQL mengecil, karena respons dari cache mengurangi kerja origin server pada kedua protokol. (menjawab RQ2)

## H3 • H4

---

H3. Entropi bentuk query yang lebih tinggi menurunkan cache-hit rate, dan penurunan ini lebih menekan GraphQL karena variasi selection set menghasilkan cache key (hash Automatic Persisted Queries) yang berbeda-beda. (menjawab RQ2)

H4. Bobot payload yang lebih berat (trajectory objek lintas frame) memperbesar selisih kinerja antara REST API dan GraphQL, karena biaya resolusi dan serialisasi bertambah seiring jumlah objek dan titik data yang harus dirakit server. (menjawab RQ2 dan RQ3)

## Dua tujuan (verbatim dari laporan).

- 1.** Menentukan kondisi caching, pola akses, dan bobot payload yang membuat REST API atau GraphQL paling optimal sebagai delivery layer pada pipeline inferensi computer vision, untuk meningkatkan efisiensi payload dan cache-hit rate pada aplikasi klien yang memiliki batasan bandwidth dan resource.
- 2.** Menghasilkan matriks rekomendasi arsitektural berbasis empiris yang memetakan trade-off antara efisiensi caching, penghematan bandwidth, dan compute overhead di sisi server, untuk mencegah bottleneck komputasi pada sistem serving AI berskala industri saat menghadapi variasi pola akses dan concurrent load.



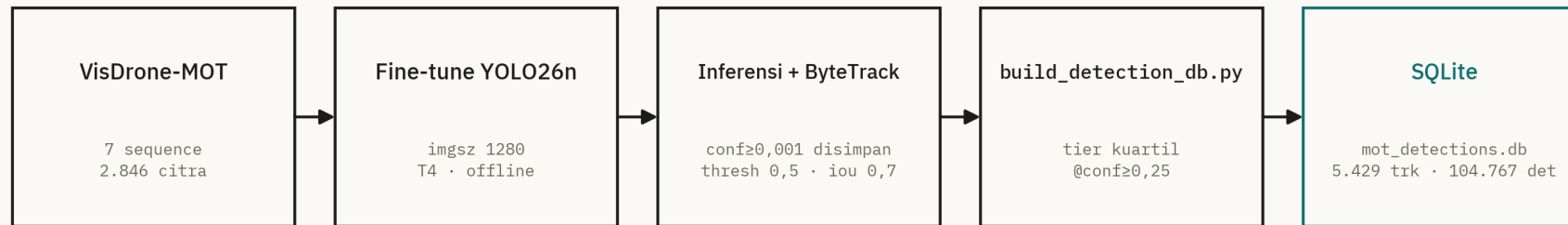
# Satu variabel bebas utama, enam metrik terikat, empat kelompok kontrol fairness.

**Variabel bebas:** jenis pendekatan API — REST (FastAPI; berbasis resource, routing statis) vs GraphQL (Strawberry; declarative data fetching, resolver traversal); eksperimen utama menambah caching, pola akses, entropi query, dan bobot payload.

**Variabel terikat:** latency (p50/p95/p99), throughput, payload size, cache-hit rate, CPU usage, memory usage.

**Variabel kontrol:** mesin inferensi beku (YOLO26 NMS-free, deterministik); shared DetectionDAL di atas SQLite; header cache dari satu fungsi shared + APQ-over-GET; isolasi cgroup/netns, HTTP/1.1 tanpa kompresi, 1 worker.

# Satu pipeline luring membekukan korpus: 7 sequence · 2.846 image · 5.429 track · 104.767 detection.



satu kali, luring — model tidak pernah dieksekusi saat pengujian beban

## Perangkat lunak & alat pengujian (Tabel III.3–III.4).

| Komponen              | Versi            | Peran                              |
|-----------------------|------------------|------------------------------------|
| Ubuntu Linux (VM KVM) | kernel 6.8.0-124 | netns, netem, systemd-run (cgroup) |
| Python                | 3.12.3           | bahasa implementasi APE            |
| YOLO26 (Ultralytics)  | terkini          | inference engine NMS-free          |
| FastAPI               | 0.137.2          | framework REST API                 |
| Strawberry GraphQL    | 0.317.2          | library GraphQL                    |
| Grafana k6            | 2.0.0            | load testing (P95, throughput)     |
| psutil                | 7.2.2            | polling CPU & memori               |
| Varnish               | –                | cache layer / reverse proxy (VCL)  |
| Network namespace     | –                | profil lan / constrained via netem |

Versi persis kolom 'Versi' Tabel III.4; Varnish & netns tidak diberi nomor versi di tabel laporan (ditulis apa adanya).

# Kontribusi utama: metodologi benchmark yang adil beserta instrumen APE yang dapat direplikasi — bukan semata angka perbandingan.

**Produk akhir:** matriks rekomendasi arsitektural dan blueprint pemilihan protokol API yang memetakan trade-off efisiensi transfer data vs utilisasi resource server, beserta validasi arsitektur pengujian yang mengisolasi bottleneck basis data.

**Untuk praktisi ML serving:** kepastian teknis mengenai kondisi densitas objek dan tingkat concurrency di mana fleksibilitas GraphQL mulai membebani compute overhead server dibandingkan REST.

**Untuk peneliti:** instrumen APE — shared data-access layer + cache layer identik untuk kedua protokol — sebagai acuan desain eksperimen yang lebih fair pada payload nested khas computer vision.

## Tujuh fase berurutan, dirancang untuk validitas, reliabilitas, dan keterulangan (reproducibility).

**Fase 1–3:** studi literatur → identifikasi masalah → perancangan penelitian.

**Fase 4:** pembangunan aplikasi pendukung eksperimen (APE) — instrumen fairness dua-protokol.

**Fase 5–7:** pelaksanaan eksperimen → analisis data eksperimen → penyimpulan eksperimen.

**Kontrak per fase:** setiap fase memiliki luaran spesifik untuk mengisolasi variabel bebas secara ketat.

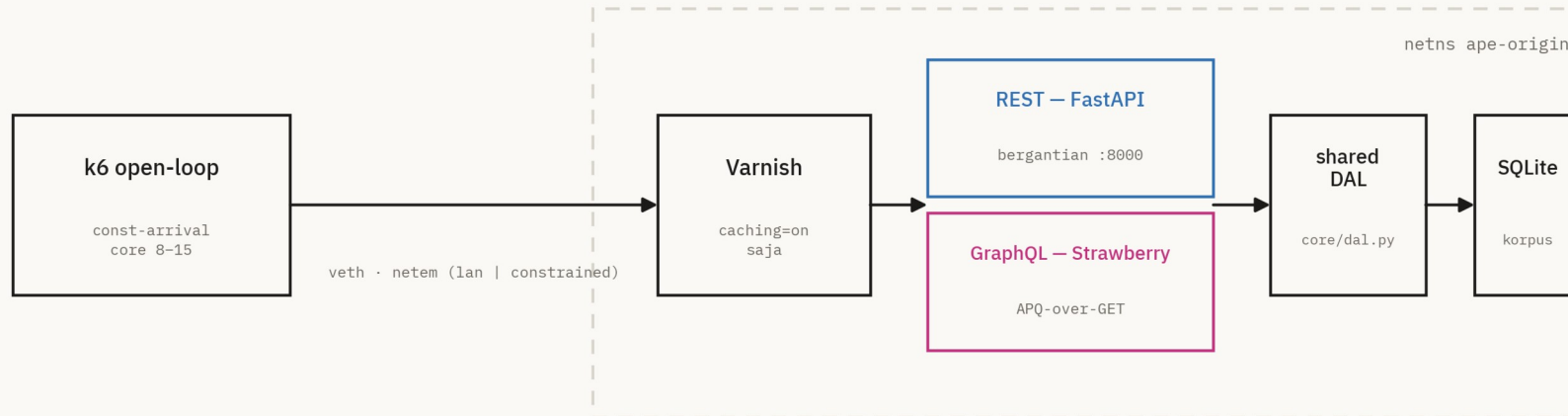
# Perubahan scope terdokumentasi: uji pendahuluan in-memory (DET) → eksperimen utama korpus MOT relasional.

**Keputusan metodologis (Batasan, verbatim inti):** SQLite mot\_detections.db + shared DAL  
“menggantikan rancangan uji pendahuluan yang sebelumnya tidak menggunakan basis data (in-memory data pool), karena rancangan tersebut tidak dapat merepresentasikan kondisi caching maupun pola akses data yang realistis.”

**Mengapa VisDrone-MOT (bukan DET):** anotasi track lintas frame membentuk payload berat (heavy) secara natural, bukan disimulasikan.

**Batasan lain:** akurasi (mAP) bukan variabel; hanya REST & GraphQL; API read-only tanpa autentikasi; jaringan lan/constrained via netem.

# Fairness by construction: kedua protokol melewati jalur data, header cache, dan lingkungan yang identik — dibuktikan, bukan diasumsikan.



`systemd-run scope: CPUQuota 400% · MemoryMax 2048M · server core 0-7 · sampler core 31`

satu server hidup pada satu waktu — REST dan GraphQL tidak pernah co-resident

Acceptance test A1 (paritas deep-equal) · A2 (diff SQL, APE\_LOG\_SQL) · A3 (fairness cache Varnish nyata) — lolos; APQ-over-GET membuat GraphQL cacheable setara REST.

## Dua korpus saling melengkapi: 3.240 run skenario MOT + 720 run grid caching; dianalisis non-parametrik per sel.

**Studi skenario MOT (RQ1 + RQ2 round-trip):** 2 protokol × 6 skenario (M1–M6) × 3 tier × 3 laju × 30 run = 3.240 run terukur; laju dikalibrasi 40/80/120% dari ceiling protokol terendah; r120\_overload dianalisis terpisah.

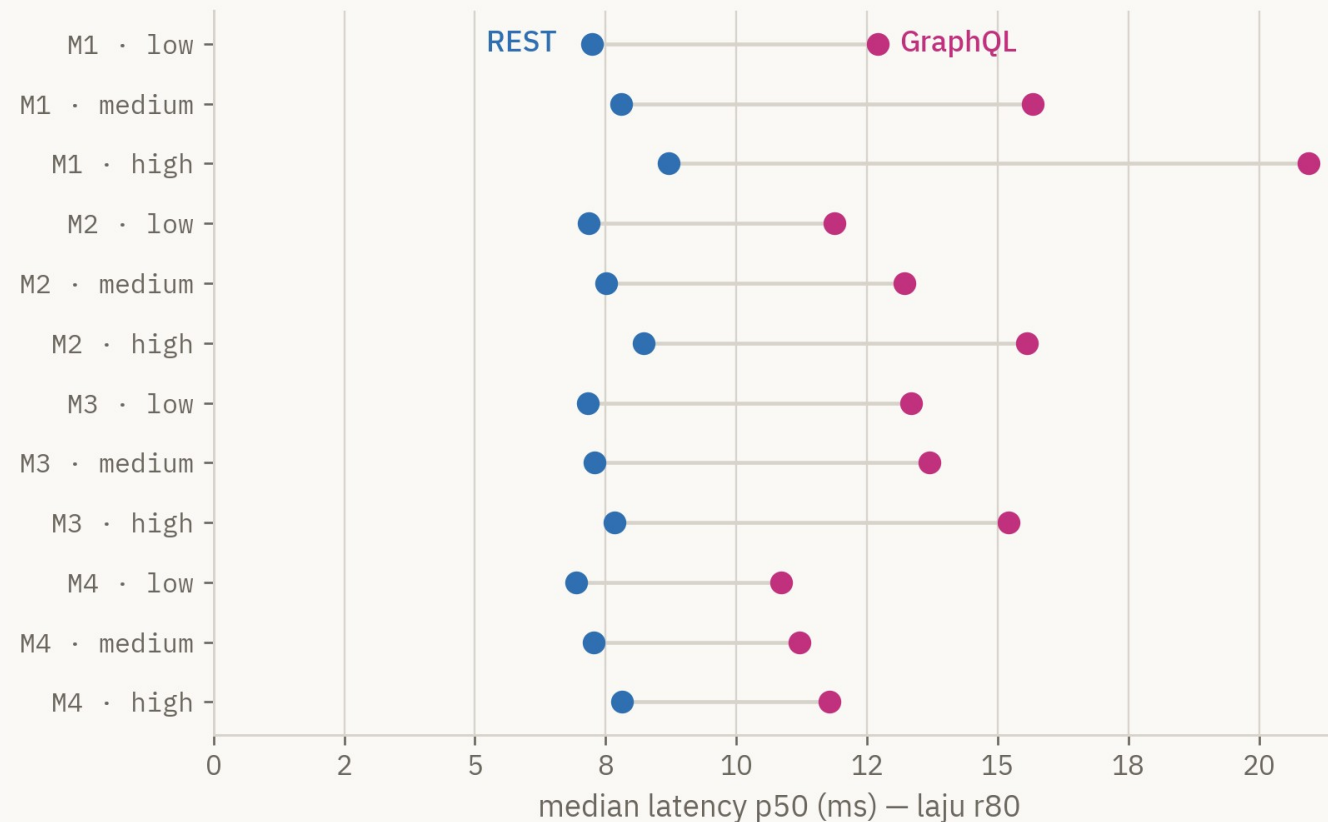
**Grid inti caching (RQ2 caching + RQ3):** 2 × 2 caching × 3 pola akses × 2 bobot payload = 24 blok × 30 run = 720 run (phase2-core-real; serial penuh).

**Analisis:** Shapiro-Wilk → Mann-Whitney U per sel + effect size rank-biserial  $\equiv$  Cliff's  $\delta$  ( $\delta < 0$  = REST unggul) → koreksi Holm per keluarga metrik; bermakna hanya jika  $p < 0,05$  DAN magnitudo  $\geq$  sedang.

**Protokol run:** 1 warm-up dibuang + 30 run terukur 90 s per sel; open-loop constant-arrival-rate; seed 42.

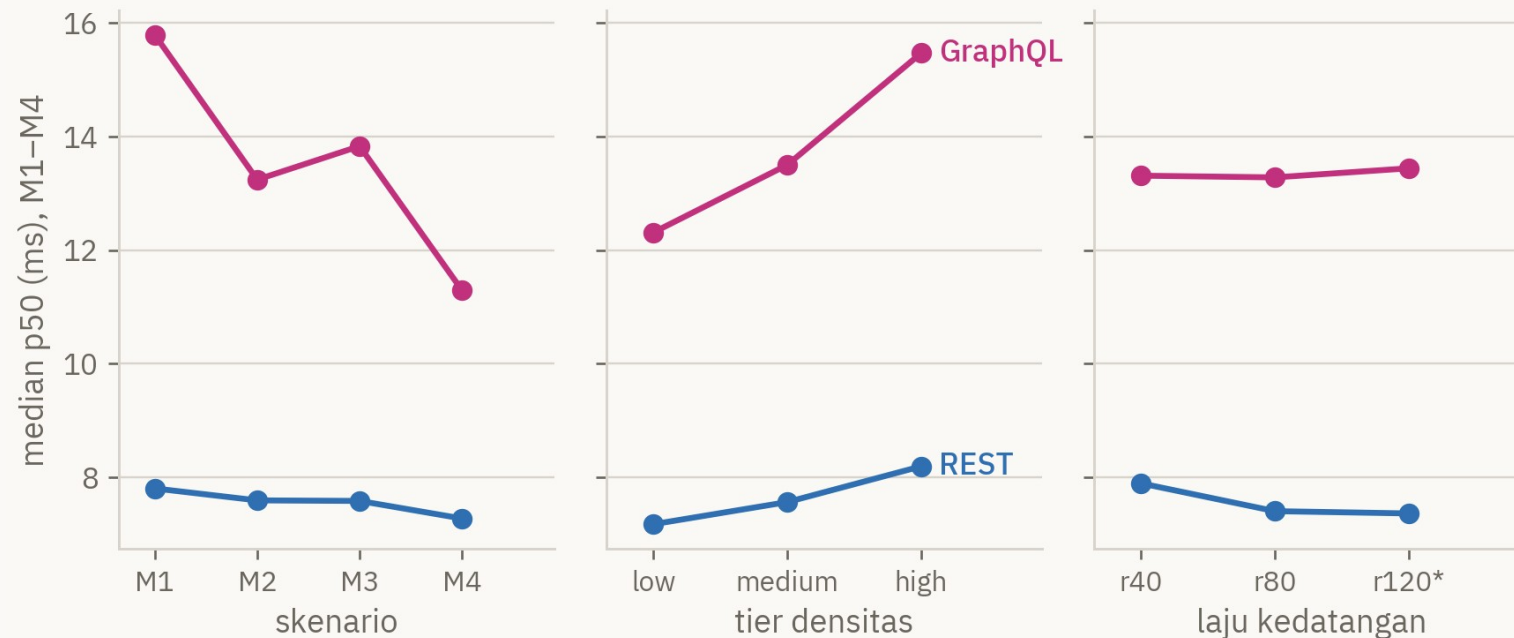


**REST unggul di seluruh 36/36 sel M1–M4 dengan pemisahan sempurna ( $\delta = -1,00$ ): +51% s.d. +141% pada r80.**



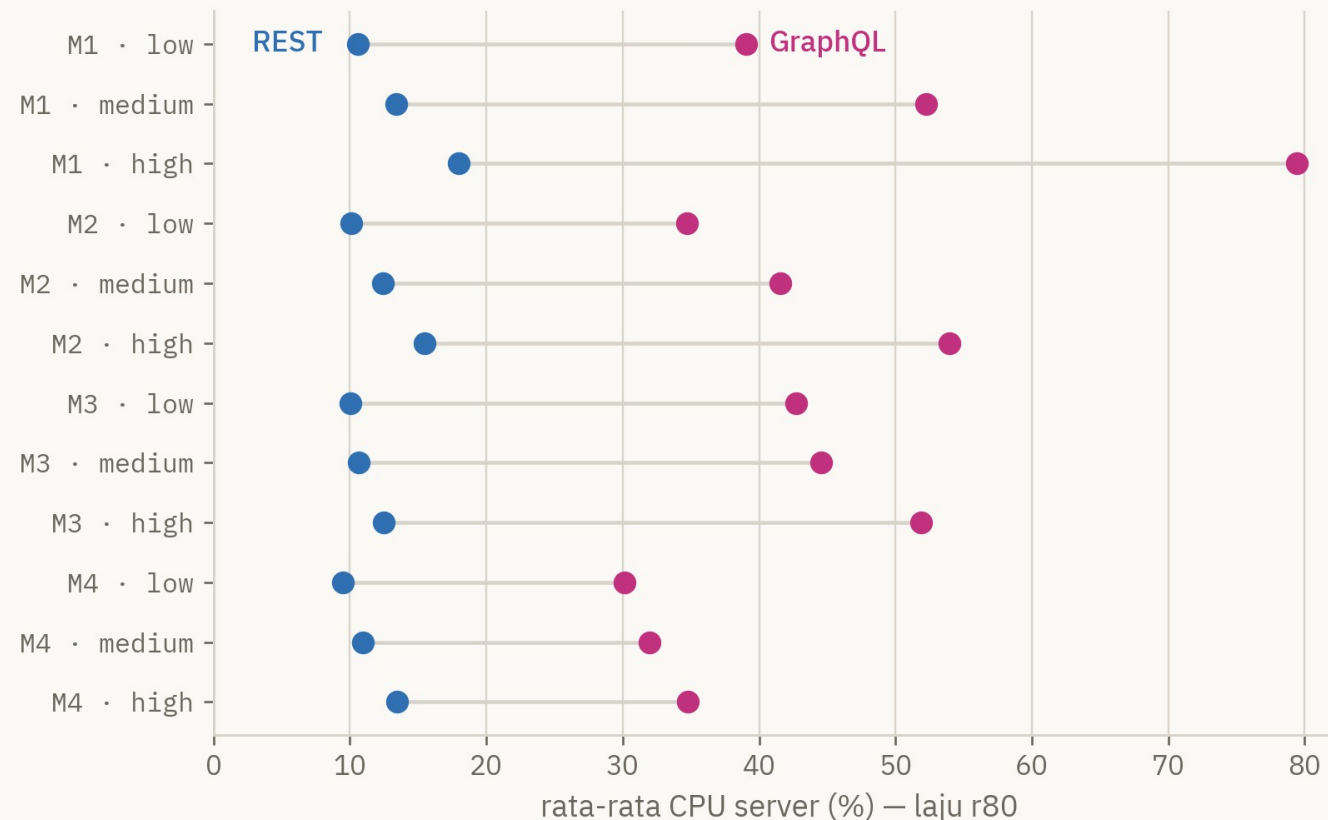
Contoh sel terpadat M1·high r80: REST 8,70 ms vs GraphQL 20,94 ms. Mendukung H1.

**Densitas objek — bukan laju kedatangan — yang memperlebar selisih: biaya GraphQL bersifat per-objek-terserialisasi.**



Median p50 low→high: GraphQL 12,3→15,5 ms vs REST 7,2→8,2 ms; laju sub-saturasi hampir tak menggeser selisih.

## Keunggulan latensi REST tidak dibayar di tempat lain: GraphQL memakai $\pm 4\times$ CPU untuk kerja yang sama.



M1·high r80: CPU REST 17,96% vs GraphQL 79,45% ( $\approx 4,4\times$ ) — biaya parsing kueri, validasi skema, resolver traversal. Tervalidasi ulang di grid caching (Tabel L.2, phase2-core-clean).

**Saat laju melampaui titik jenuh, GraphQL kolaps lebih dulu — dua orde besaran di atas REST pada laju yang sama.**

**6.725 ms**

median p50 GraphQL — M1-high pada r120 (74 iter/s)

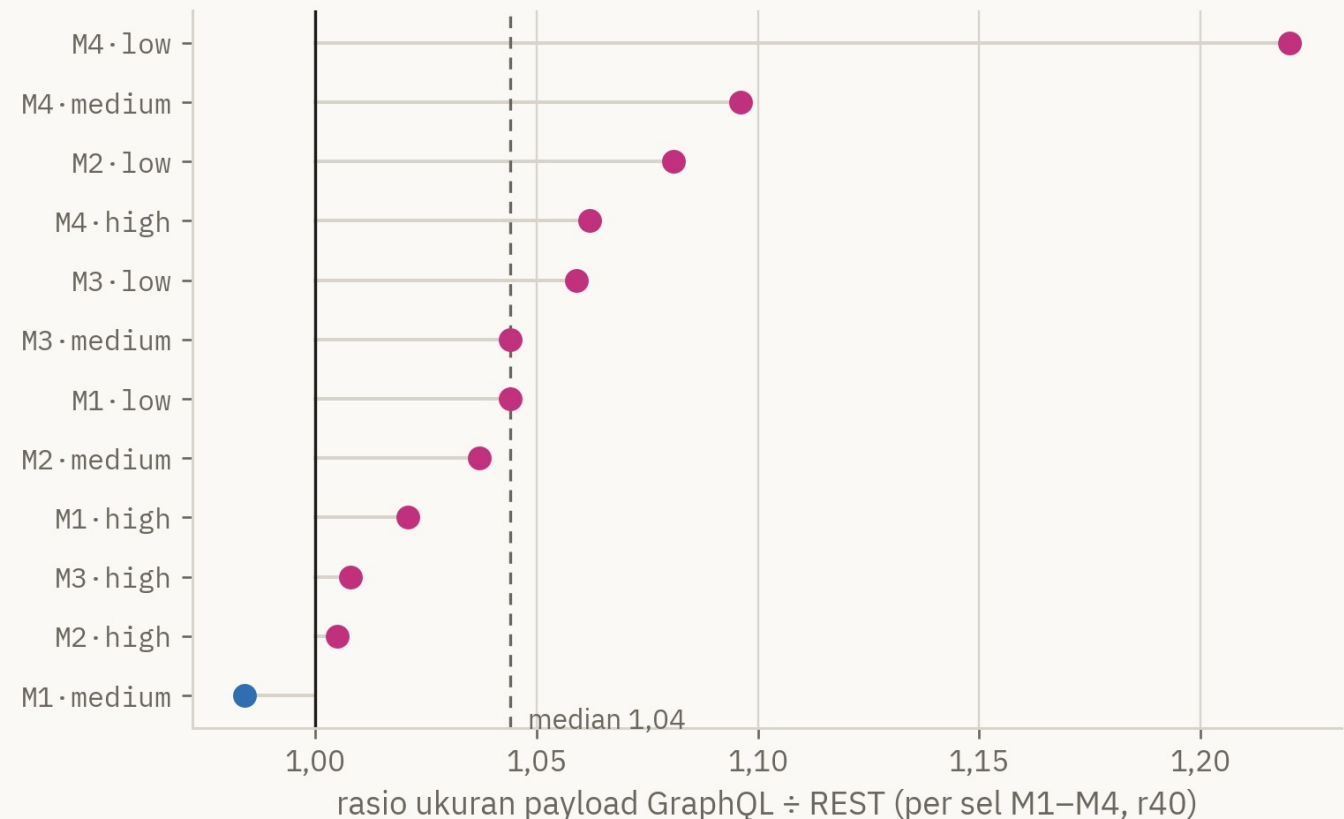
**8,64 ms**

median p50 REST — sel dan laju yang sama

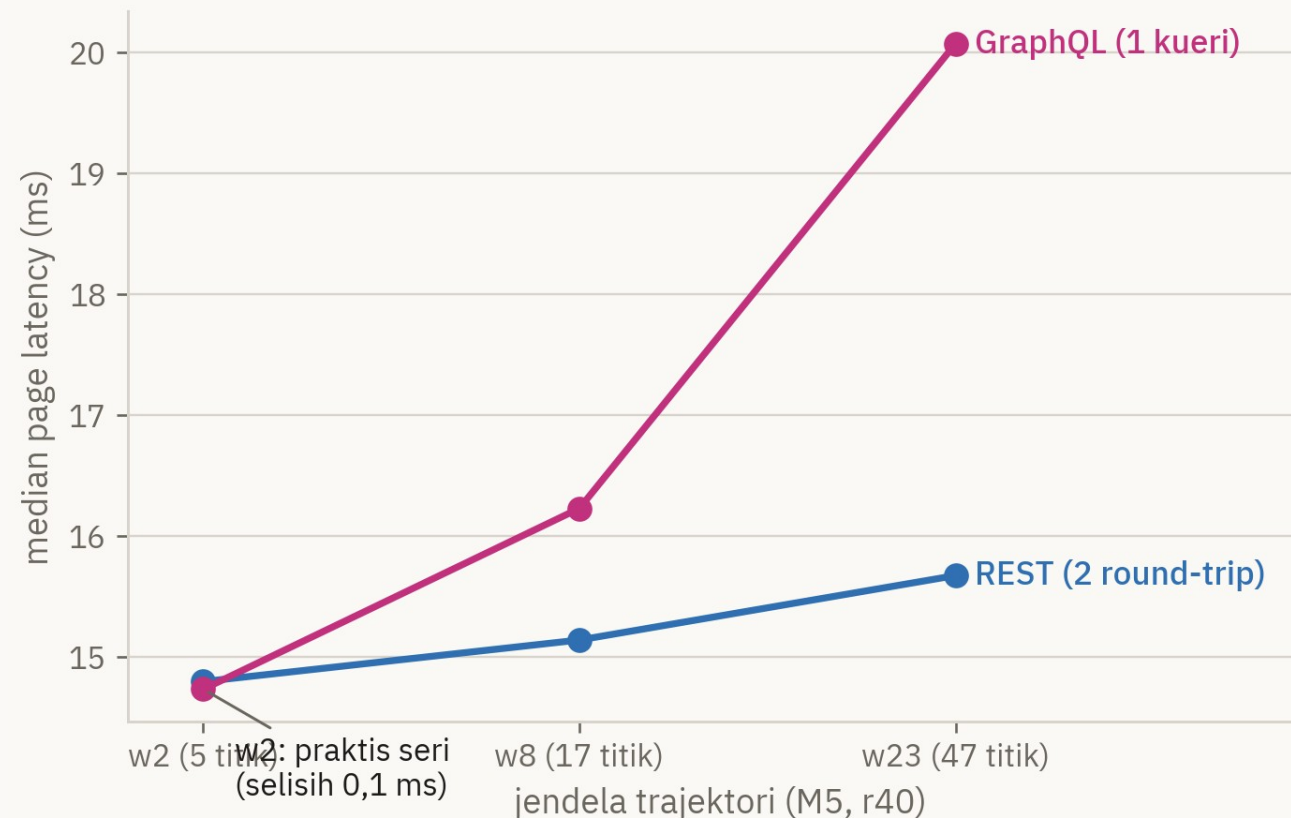
# Anti-over-fetching GraphQL tidak terbukti pada API yang dirancang setara – dan selisih throughput tidak bermakna praktis.

**Payload:** rasio median GraphQL÷REST 1,04; selisih  $\pm 30$  byte = amplop {"data":...}; REST lebih kecil di 46/48 sel factorial-A.

**Throughput:** selisih median  $< 0,03\%$  pada sel sebanding (39 sel) – padahal  $\delta$  latensi terkunci  $-1,00$ : signifikansi statistik  $\neq$  signifikansi praktis.

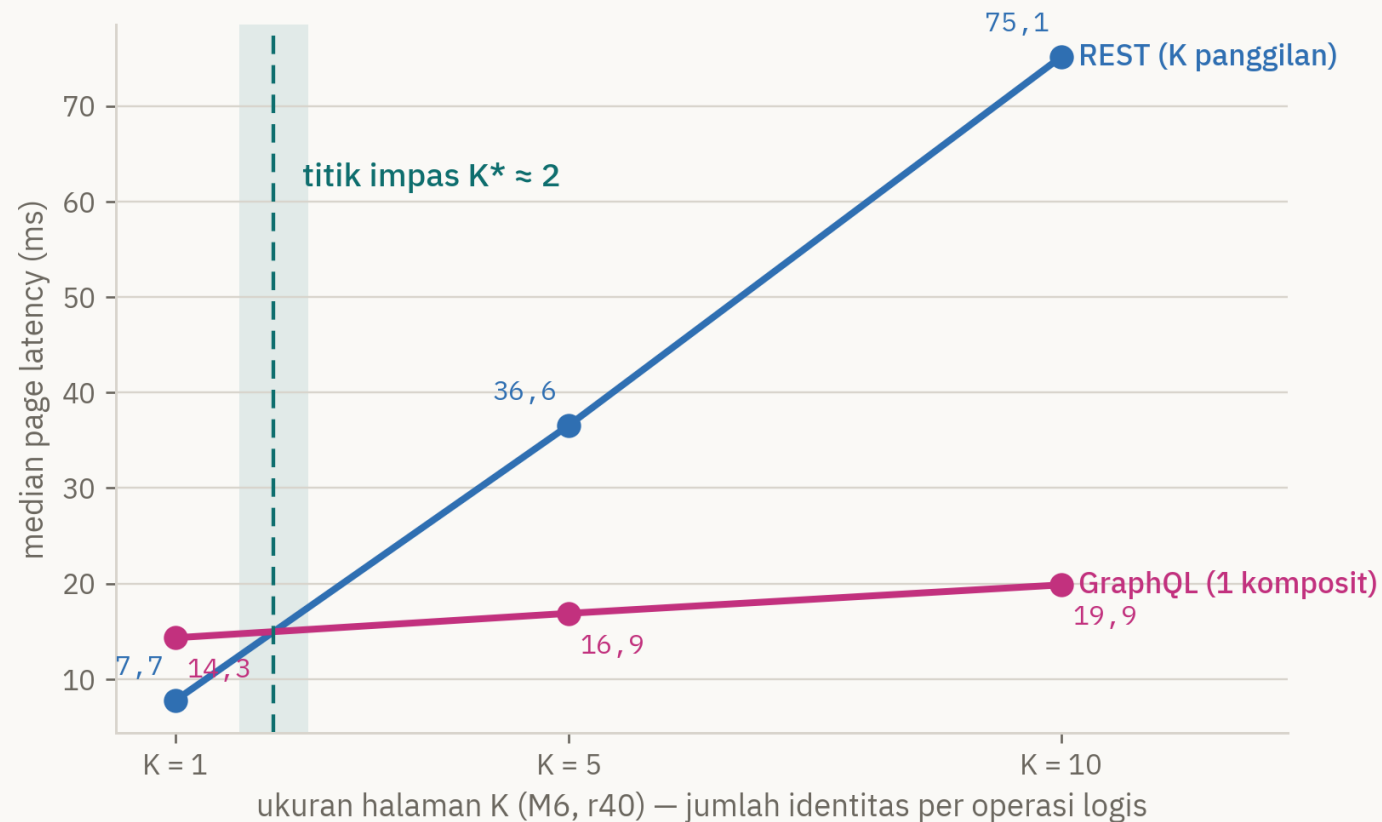


**Menghemat satu round-trip belum membalik keadaan: GraphQL hanya seri di jendela terkecil, kalah saat jendela membesar.**



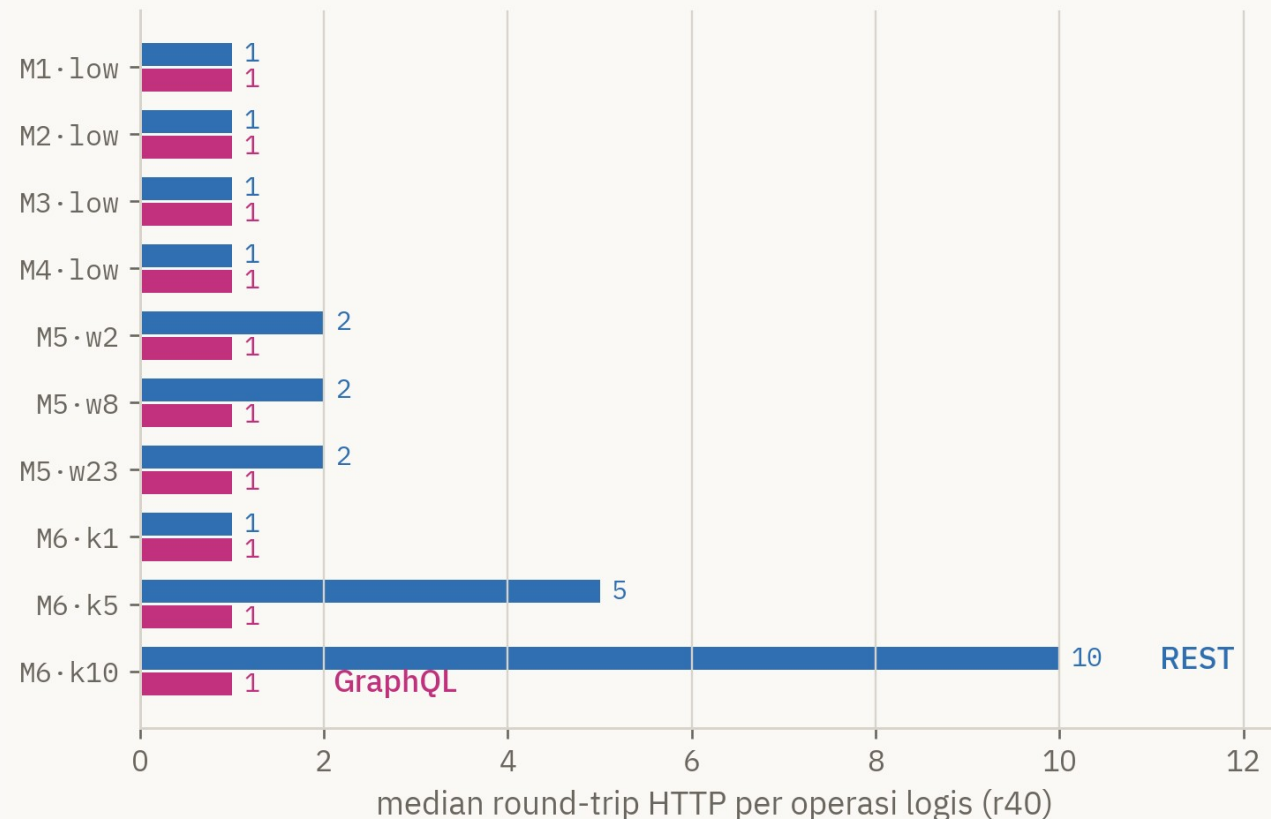
w2: 14,73 vs 14,79 ms (satu-satunya sel imbang sejati;  $\delta +0,32$ ; selisih 0,1 ms) · w23: 20,07 vs 15,67 ms.

**Pemenang berbalik di fan-out: titik impas  $K^* \approx 2$ , dan pada  $K \geq 5$  GraphQL menang penuh.**



K=1: 7,75 vs 14,33 · K=5: 36,57 vs 16,88 · K=10: 75,13 vs 19,86 ms (REST vs GraphQL). Satu-satunya kelas skenario di mana GraphQL konsisten mengungguli REST.

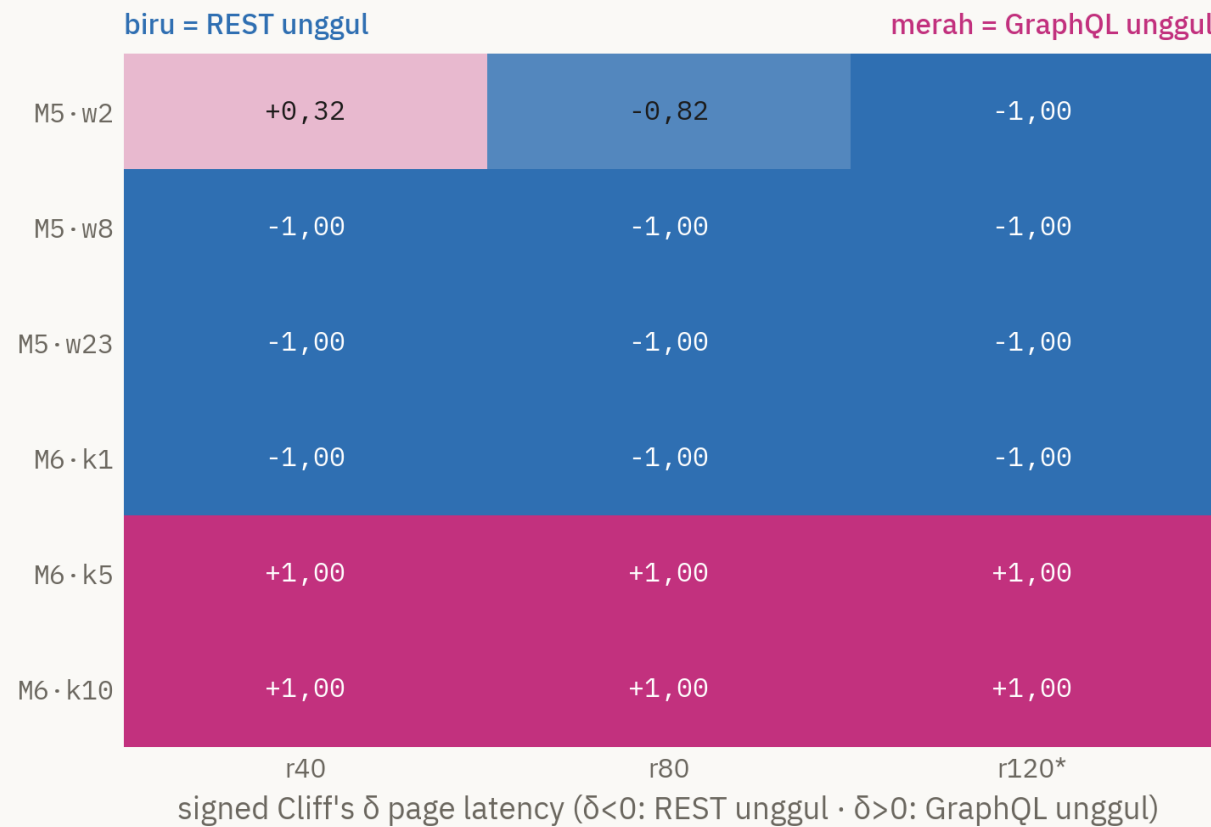
## Mengapa berbalik? Bukan karena GraphQL lebih cepat per-permintaan — REST membayar K perjalanan bolak-balik.



Ekonomi round-trip murni: keunggulan fan-out tinggi bersumber dari penghematan  $K-1$  round-trip, bukan kecepatan per-permintaan.



**Satu peta merangkum semua: hanya M6 K  $\geq 5$  yang merah penuh; satu selimbang sejati (M5·w2·r40).**



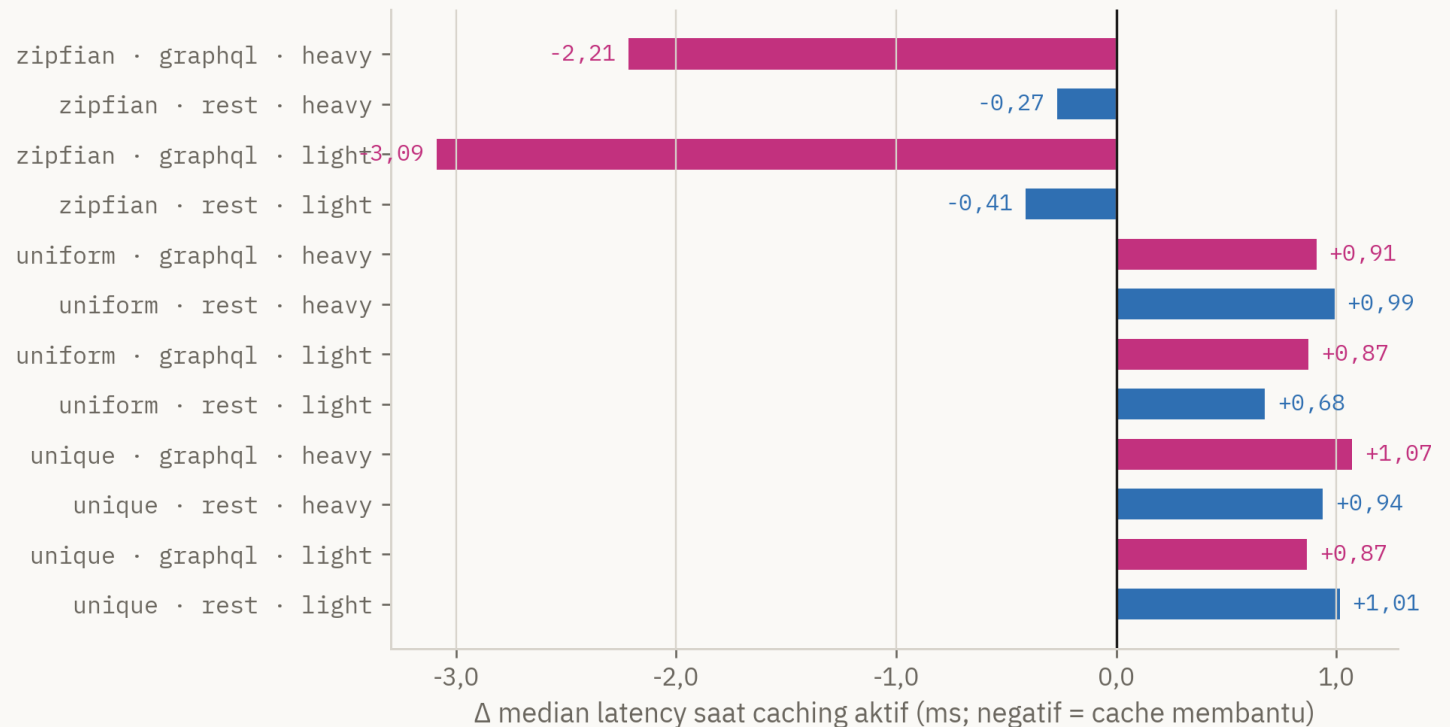
Signed Cliff's  $\delta$  page latency per sel tier  $\times$  laju.

# Lapisan cache adil — dan manfaatnya asimetris: lebih besar bagi GraphQL, bersyarat pola akses berulang.

**Fairness (N4):** hit rate zipfian·light REST 0,43 vs GraphQL 0,42 — ditentukan pola akses, bukan protokol; unique = 0,00.

**Asimetri:** zipfian·light: GraphQL -3,09 ms (34,91→31,82) vs REST -0,41 ms (28,45→28,04) — tiap hit menghindari kerja origin GraphQL yang lebih mahal.

**Syarat:** pola uniform (reuse rendah): overhead proxy justru +0,68 s.d. +0,99 ms. H2 didukung sebagian.



**Tidak ada satu pun kombinasi caching × pola akses × payload yang membalik pemenang latensi: REST  $\delta = -1,00$  di 12/12 blok.**

|                   | biru = REST unggul |         |         | merah = GraphQL unggul |         |          |
|-------------------|--------------------|---------|---------|------------------------|---------|----------|
|                   | lat_p50            | lat_p95 | lat_p99 | thruput                | payload | hit rate |
| off·uniform·heavy | -1,00              | -1,00   | -1,00   | +0,50                  | +1,00   | —        |
| off·uniform·light | -1,00              | -1,00   | -1,00   | +0,21                  | +0,96   | —        |
| off·unique·heavy  | -1,00              | -1,00   | -1,00   | +0,49                  | +1,00   | —        |
| off·unique·light  | -1,00              | -1,00   | -1,00   | +0,23                  | +0,20   | —        |
| off·zipfian·heavy | -1,00              | -1,00   | -1,00   | +0,37                  | +1,00   | —        |
| off·zipfian·light | -1,00              | -1,00   | -1,00   | +0,24                  | -0,98   | —        |
| on·uniform·heavy  | -1,00              | -1,00   | -1,00   | +0,26                  | +1,00   | -0,22    |
| on·uniform·light  | -1,00              | -1,00   | -1,00   | +0,26                  | +0,92   | +0,10    |
| on·unique·heavy   | -1,00              | -1,00   | -1,00   | +0,37                  | +1,00   | -0,01    |
| on·unique·light   | -1,00              | -1,00   | -1,00   | +0,32                  | +0,20   | +0,07    |
| on·zipfian·heavy  | -1,00              | -1,00   | -1,00   | +0,34                  | +1,00   | -0,18    |
| on·zipfian·light  | -1,00              | -1,00   | -1,00   | +0,23                  | -0,87   | +0,13    |

signed Cliff's  $\delta$  per metrik (grid caching 24 blok; — = tidak berlaku)

$\delta$  hit-rate  $-0,22 \dots +0,13$  ( $\approx 0$ : cache adil antar-protokol); caching mempersempit, tidak menghapus, selisih.

# Dua insiden ditemukan, diisolasi, dan diperbaiki – kontrol kualitas data, bukan kecelakaan yang dibiarkan.

**run-sesi-1 (uji pendahuluan):** 345.879 error serialisasi in-band GraphQL (HTTP 200 → tak terlihat error\_rate) → sesi DIDEMOSI; diganti factorial-A (2.880 run, audit bersih); guard permanen: k6 body-check “no graphql errors”.

**Grid caching phase2:** 92,6% baris sesi phase2 lain terkumpul saat 2–8 sesi paralel → RQ3 memakai HANYA phase2-core-real (720 run, serial penuh).

**Telemetri CPU/RSS:** sampler sempat memantau PID pembungkus → diperbaiki sebelum studi MOT; re-run phase2-core-clean mereplikasi 12/12 blok ( $\delta -1,00$ ; pergeseran median  $\leq 0,25$  ms).

**Ancaman lain (diakui):** netem = emulasi, bukan WAN; 1 mesin · 1 stack (FastAPI+Strawberry) · 7 sequence; profil lan vs constrained → dua korpus tak dibandingkan langsung.

# REST unggul default pada kondisi yang diuji; keunggulan GraphQL struktural dan kondisional; caching mempersempit, tidak menghapus.

**RQ1:** median latency REST konsisten lebih rendah + CPU jauh lebih kecil ( $\approx 4\times$  di sel terpadat) di seluruh M1–M4; selisih melebar seiring densitas; overload menegaskan. H1 didukung.

**RQ2:** konsolidasi round-trip termonetisasi hanya bila biaya resolusi  $<$  penghematan round-trip; pada LAN teremulasi ( $\approx 5$  ms) crossover cepat: M5 seri di w2, kalah di w23; M6 menang penuh di  $K \geq 5$  ( $K^* \approx 2$ ).

**RQ3:** APQ-over-GET menyetarakan cacheability (hit rate setara); manfaat cache asimetris pro-GraphQL & bersyarat akses berulang. H2 didukung sebagian; H3 konsisten mekanistik; H4: [VERIFY: vonis H4 belum eksplisit di Bab V/VII].

**Kontribusi:** metodologi benchmark fair + instrumen APE replikabel — hasil scoped, bukan klaim universal.

DEMO

**Instrumen yang menghasilkan seluruh angka pada deck ini —  
dijalankan langsung.**

# Demo

Alat Pembantu Eksperimen (APE) — REST · GraphQL · shared DetectionDAL · mot\_detections.db

# Terima kasih – siap menerima pertanyaan.

**Saran:** validasi WAN nyata (bukan hanya emulasi netem); pengujian dengan autentikasi/otorisasi; sub-studi trade-off batching GraphQL (APE\_GRAPHQL\_BATCHING on/off); replikasi pada corpus deteksi objek lain.

**Keberlanjutan:** paket riset replikabel (dokumentasi + skrip corpus + orkestrator), dashboard hasil, evaluasi rilis corpus turunan sesuai lisensi VisDrone.

**Jeihan Ilham Kusumawardhana – 221524042**

# 1

## Lampiran — Konteks & Latar Belakang



# Latar belakang lengkap: dari inferensi real-time ke lapisan penyajian data.

**Deteksi objek:** YOLO26 end-to-end bebas NMS — standar industri untuk real-time (Redmon et al. 2016; Sapkota & Karkee 2025); keluaran = metadata nested: class, confidence, bounding box, track (Nieto et al. 2021).

**Masalah penyajian:** penyajian lambat menggerus manfaat inferensi real-time meski akurasi model tinggi.

**Dua tahap:** uji pendahuluan in-memory → temuan: tanpa DB tidak realistis & caching belum terkontrol → eksperimen utama corpus relasional + caching sebagai variabel.

**Tujuan:** kondisi caching/pola akses/bobot payload yang membuat REST atau GraphQL paling optimal + matriks rekomendasi arsitektural berbasis empiris.

# Pivot dua tahap adalah kejujuran metodologis: pilot mengungkap confounder, eksperimen utama memperbaikinya.

**Temuan pilot (Bab V):** (1) rancangan tanpa basis data tidak merepresentasikan pola akses realistis; (2) caching — pembeda paling signifikan REST vs GraphQL di praktik — belum dikontrol sebagai variabel.

**Audit teknis pilot:** selisih besar didominasi mode implementasi typed vs passthrough, bukan protokol; bug GraphQL-passthrough & asimetri konstruksi partial ditemukan dan diperbaiki (commit 824b195).

**Posisi pilot:** kalibrasi instrumen — paritas data terkonfirmasi (A1); tabel latensinya tidak dipakai sebagai hasil.

# Rumusan masalah (verbatim Bab I §I.2).

Belum adanya bukti empiris yang membandingkan secara fair kinerja antara arsitektur REST API dan GraphQL dalam menyajikan data deteksi objek ketika kedua protokol menggunakan sumber data, jalur akses data, serta konfigurasi caching yang setara.

Ketidakpastian mengenai seberapa besar pengaruh caching, pola akses data, dan entropi bentuk query terhadap latency, throughput, dan cache-hit rate pada REST API dibandingkan dengan GraphQL, terutama karena GraphQL secara default tidak cacheable melalui mekanisme HTTP standar.

Belum teridentifikasinya secara komprehensif titik trade-off antara efisiensi transfer data (ukuran payload), cache-hit rate, dan utilisasi sumber daya komputasi server ketika kedua arsitektur API dihadapkan pada variasi bobot payload, kepadatan objek, dan beban akses konkuren.

# 2

## Lampiran — Training & Model

## Konfigurasi fine-tuning YOLO26 (Tabel III.2) + split data.

| Parameter             | Nilai                       | Catatan                   |
|-----------------------|-----------------------------|---------------------------|
| Pendekatan            | Fine-tuning dari bobot COCO | bukan from scratch        |
| Ukuran model          | YOLO26n (nano)              | mulai dari model terkecil |
| Resolusi (imgsz)      | 1280                        | objek kecil citra drone   |
| Freeze layer          | None                        | domain jauh dari COCO     |
| Optimizer             | SGD lr0=0,01 momentum=0,937 | eksplisit – bukan 'auto'  |
| Epochs / patience     | 200 (plafon) / 15           | plafon naik dari 100      |
| Batch size            | 4                           | T4 16GB @1280, amp        |
| Seed / deterministic  | 42 / False                  |                           |
| Split resmi (run3)    | train 6.471 / val 548       | protokol VisDrone-DET     |
| Split internal (run2) | train 5.824 / val-int 647   | 90/10 dr 6.471 (derivasi) |
| Checkpoint korpus     | yolo26n_t4_run2/best.pt     | lihat catatan di notes    |

Kriteria penerimaan dua lapis: konvergensi early-stopping + kecukupan densitas keluaran (anti-sirkularitas: tier kuartil mekanis).

# Pilot optimizer: 'auto' memilih MuSGD — $\pm 4\times$ lebih lambat per iterasi di T4; SGD eksplisit dikunci.

**Gejala:** pelatihan final berjalan 1,2 detik/iterasi ( $\approx 4\times$  lebih lambat dari baseline); plafon 200 epoch  $\approx$  lima hari.

**Eliminasi sistem:** versi pustaka, driver GPU, kernel, link PCIe, throttling termal, kuota CPU, I/O disk — semuanya tidak berubah / bukan bottleneck; profil proses: assigner di CPU, GPU mayoritas idle.

**Uji A/B satu-variabel:** optimizer='auto' memilih MuSGD saat total iterasi melebihi ambang; MuSGD  $\approx 4\times$  lebih lambat dari SGD di T4; uji singkat lolos karena epoch kecil memilih AdamW.

**Keputusan:** SGD eksplisit (lr=0,01, momentum=0,937) — stabil 3,6–4,1 iterasi/detik ( $\approx 28$  jam plafon 200 epoch).

# Ledger deviasi run3 (Kaggle, protokol resmi) — korpus eksperimen tidak tersentuh.

**Protokol resmi:** train = 6.471 (scan: '6448 images, 23 backgrounds'), val = 548; args identik train\_yolo26.py run3; aktual berjalan single-T4 (device '0', batch 4 → nol deviasi batch).

**Deviasi tercatat:** workers 4 (Kaggle 4 vCPU); time=10.0 jam/sesi (commit sebelum hard-kill 12 jam); mirror val cacat 150/548 → val lengkap dibawa sendiri.

**Hard rule:** mot\_detections.db FROZEN — tidak ada satu pun keluaran run3 yang meregenerasi korpus eksperimen.

# 3

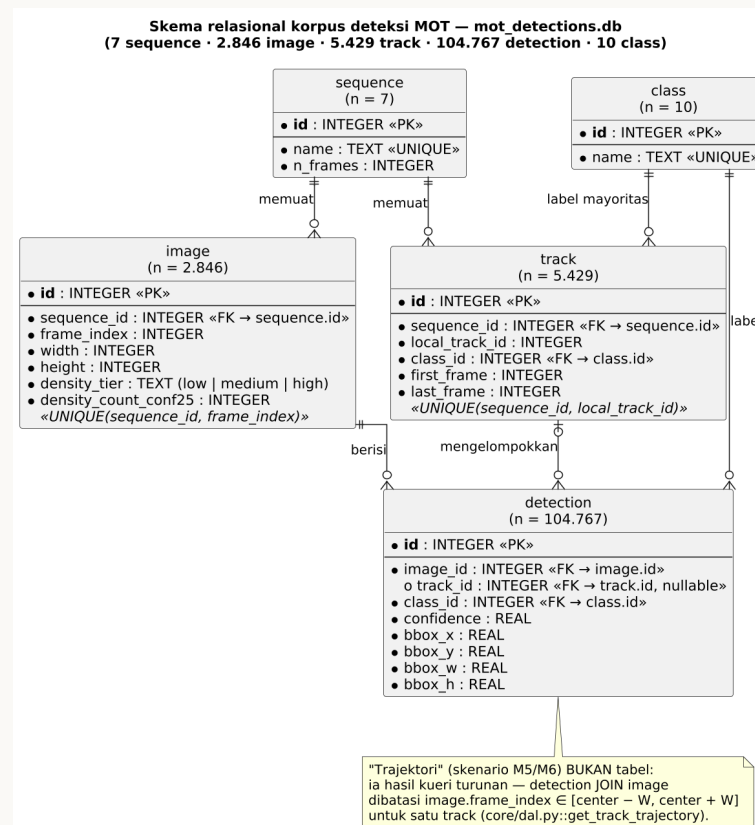
## Lampiran — Dataset & Korpus



## Statistik korpus mot\_detections.db (diverifikasi query langsung).

| Besaran              | Nilai                                              | Keterangan                                         |
|----------------------|----------------------------------------------------|----------------------------------------------------|
| Sequence             | 7                                                  | VisDrone-MOT val                                   |
| Image                | 2.846                                              |                                                    |
| Track                | 5.429                                              | 0 baris track_id NULL                              |
| Detection            | 104.767                                            | conf $\geq$ 0,001 disimpan                         |
| Kelas                | 10                                                 | pedestrian...motor                                 |
| Objek/citra min-maks | 1 – 134                                            |                                                    |
| Mean / median        | 36,53 / 36,0                                       |                                                    |
| Std dev              | 32,31                                              |                                                    |
| Q1 / Q3              | 5 / 54                                             | ambang tier kuartil                                |
| Tier low/medium/high | 579 / 1.555 / 712                                  | 20,3% / 54,6% / 25,0%                              |
| Top-5 kelas          | pedestrian 37.166 (35,5%)<br>people 13.254 (12,7%) | car 29.609 (28,3%)<br>motor 10.445 · bicycle 5.545 |

# Skema relasional: sequence — image — detection — track — class; trajectory adalah kueri turunan, bukan tabel.



Definisi korpus (build\_detection\_db.py): conf ≥ 0,001 disimpan; tier densitas dihitung @conf ≥ 0,25; kelas track = majority vote; kunci track global (sequence, target\_id).

# Asal track\_id: ByteTrack (inferensi model) — dibuktikan dengan perbandingan set ID per sequence, bukan asumsi.

**Skrip:** infer\_mot\_track.py — “self-consistent track IDs from the model itself, rather than stitching GT tracks”; tracker Ultralytics ByteTrack.

**Parameter:** conf 0,25 (default inferensi; korpus memakai arm conf001), track\_high\_thresh 0,5, iou 0,7, imgsz 1280.

**Bukti data (2026-07-13):** 7/7 sequence — set (sequence, target\_id) pada txt ByteTrack  $\equiv$  set (sequence, local\_track\_id) di DB, dan jumlah baris deteksi per sequence identik (total 5.429 track, 104.767 baris).

**Konsekuensi:** seluruh 104.767 deteksi memiliki track (0 NULL); skema tetap mengizinkan NULL untuk kotak yang tidak ter-track.

# 4

## Lampiran — Lingkungan & Beban

# Perangkat keras & lunak (Tabel III.2–III.4).

| Komponen           | Nilai                                          |
|--------------------|------------------------------------------------|
| CPU                | AMD EPYC 7302 (KVM) · 32 vCPU daring (0–31)    |
| Pinning            | server 0–7 · k6 8–15 · sampler 31              |
| RAM                | 64 GB (64.276 MB) · swap 8 GB                  |
| GPU                | NVIDIA Tesla T4 16 GB – luring, mesin terpisah |
| Penyimpanan        | SSD NVMe 512 GB                                |
| OS                 | Ubuntu Linux, kernel 6.8.0-124-generic (KVM)   |
| Python             | 3.12.3                                         |
| FastAPI            | 0.137.2                                        |
| Strawberry GraphQL | 0.317.2                                        |
| Grafana k6         | 2.0.0                                          |
| psutil             | 7.2.2                                          |
| Jaringan           | network namespace (lan   constrained) via veth |

# Isolasi resource: kuota seragam + pinning + server bergantian — kontensi tidak menjelaskan hasil.

**systemd-run scope:** CPUQuota 400% · MemoryMax 2048M per server; scope terpisah per sesi.

**Pinning (taskset):** server core 0–7, k6 core 8–15, sampler telemetry core 31 — terverifikasi live saat launch phase2-core-clean.

**Serial ketat:** satu sel pada satu waktu; server REST/GraphQL bergantian di port 8000, tidak pernah co-resident; urutan blok diacak seed 42 (anti drift termal).

# Profil netem & topologi: satu hop klien↔edge yang ditunda — bukan seluruh loopback.

**lan:** delay  $5\text{ ms} \pm 1\text{ ms}$  (distribusi normal), rate 100 Mbit — dipakai studi skenario MOT.

**constrained:** delay  $25\text{ ms} \pm 5\text{ ms}$  per arah ( $\text{RTT} \approx 50\text{ ms}$ ), rate 10 Mbit — dipakai grid caching.

**Perbaikan topologi:** netem hanya di sisi host veth; hop Varnish↔backend di loopback namespace tak tersentuh — artefak double-delay (MISS terukur  $2\times \rightarrow 1\times$ ) dieliminasi.

**Konsekuensi analisis:** korpus lan vs constrained tidak dibandingkan lintas korpus (diakui di Ancaman Validitas).

# Varnish: keputusan cache 100% dari header backend — protokol-agnostik by design.

**Backend tunggal 127.0.0.1:8000:** REST/GraphQL bergantian; Varnish tidak tahu protokol mana yang hidup.

**Hanya GET yang cacheable:** POST /graphql selalu pass (tidak di-cache, bukan error).

**vcl\_builtin\_backend\_response dipanggil eksplisit:** tanpa ini 'Cache-Control: no-store' DIABAIKAN diam-diam — bug nyata yang tertangkap saat verifikasi Phase 2a (endpoint random sempat HIT).

**X-Cache HIT/MISS:** satu-satunya tambahan di atas default — dibaca acceptance test A3 dan metrik cache\_hit\_rate.



# Enam skenario M1–M6 = operasionalisasi pola kebutuhan klien; k6 open-loop agar tail latency jujur.

**M1 full · M2 sparse (fields=class\_id,confidence) · M3 filter (class\_id=4 'car' 28%, min\_confidence=0,5) · M4 agregasi class\_counts.**

**M5 trajectory window:** REST 2 round-trip (track → trajectory) vs GraphQL 1 komposit; window w2/w8/w23 = 5/17/47 titik (pool 2.806/1.385/563 track).

**M6 pagination:** REST K panggilan cacheable vs GraphQL 1 komposit tracks(ids); K = 1/5/10.

**k6 constant-arrival-rate:** laju TIDAK melambat saat server melambat (anti coordinated omission — Tene 2015); summaryTrendStats p50/p95/p99.

# 5

## Lampiran — Grid & Kalibrasi

**Grid inti caching:  $2 \times 2 \times 3 \times 2 = 24$  blok  $\times$  30 run = 720 run terukur.**

| Faktor          | Taraf                               | Keterangan                        |
|-----------------|-------------------------------------|-----------------------------------|
| Protokol        | REST · GraphQL                      | variabel bebas utama              |
| Caching         | on · off                            | on: Varnish; GraphQL APQ-over-GET |
| Pola akses      | unique · uniform · zipfian          | atur reuse identitas              |
| Bobot payload   | light · heavy                       | deteksi/citra vs trajectory       |
| Entropi query   | sedang (dikunci)                    | drill-in opsional                 |
| Densitas objek  | sedang (dikunci)                    | drill-in opsional                 |
| Profil jaringan | constrained (dikunci)               | netem via netns                   |
| Laju permintaan | 10 rps (dikunci)                    | constant-arrival-rate             |
| Eksekusi        | $24 \times (1 \text{ warmup} + 30)$ | = 744 run; 720 terukur            |

## Matriks skenario MOT: 54 sel × 2 protokol × 30 run = 3.240 run terukur.

| ID | Skenario             | Tier                  | r40/r80/r120 (iter/s) |
|----|----------------------|-----------------------|-----------------------|
| M1 | Full detection fetch | densitas low/med/high | 25 / 50 / 74          |
| M2 | Selective field      | densitas low/med/high | 25 / 50 / 74          |
| M3 | Filtered detection   | densitas low/med/high | 25 / 50 / 74          |
| M4 | Aggregated           | densitas low/med/high | 25 / 50 / 74          |
| M5 | Trajectory window    | window w2/w8/w23      | 25 / 50 / 74          |
| M6 | Pagination (K RT)    | halaman k1/k5/k10     | 17 / 34 / 52          |

Setiap sel: 1 warm-up dibuang + 30 run terukur 90 s; r120\_overload dianalisis terpisah, tidak pernah dipool dengan r40/r80.

## Ceiling saturasi per keluarga → laju 40/80/120% dari protokol terendah.

| Keluarga      | Ceiling REST | Ceiling GraphQL | Terendah | r40/r80/r120 |
|---------------|--------------|-----------------|----------|--------------|
| image (M1–M4) | 100          | 62              | 62       | 25 / 50 / 74 |
| track (M5)    | 100          | 62              | 62       | 25 / 50 / 74 |
| page (M6)     | 43           | 62              | 43       | 17 / 34 / 52 |

Aturan saturasi:  $\text{dropped\_iterations} > 0 \vee \text{error} > 1\% \vee \text{p95} > \max(5 \times \text{baseline}, 150 \text{ ms})$ . GraphQL ~62 iter/s di SEMUA keluarga (framework-bound); REST page-family dibatasi fan-out K (43 hal/s  $\approx$  430 call/s).

## Dua korpus hasil — jangan bandingkan angka latensi lintas korpus.

| Aspek        | mot-scenarios-core (RQ1/RQ2) | phase2-core-real (RQ3)          |
|--------------|------------------------------|---------------------------------|
| Grid         | skenario×tier×laju (M1-M6)   | caching×akses×payload           |
| Run terukur  | 3.240                        | 720                             |
| Network      | lan                          | constrained                     |
| Caching      | off saja                     | on/off (Varnish)                |
| Pola akses   | uniform                      | uniform/zipfian/unique          |
| Laju         | kalibrasi 40/80/120%         | 10 rps tetap                    |
| Workload     | k6/workload_mot.js           | k6/workload.js                  |
| CPU/RSS      | valid                        | invalid → phase2-core-clean     |
| Env snapshot | ya                           | tidak                           |
| Serial       | ya (terverifikasi)           | ya (satu-satunya phase2 serial) |

# 6

## Lampiran — Tabel Statistik per RQ

Tabel V.1 — tier tinggi, r80: latensi & CPU per skenario.

| Skenario             | p50 REST (ms) | p50 GraphQL (ms) | CPU REST (%) | CPU GraphQL (%) |
|----------------------|---------------|------------------|--------------|-----------------|
| M1 (seluruh deteksi) | 8,70          | 20,94            | 18,0         | 79,5            |
| M2 (seleksi field)   | 8,22          | 15,57            | 15,4         | 54,0            |
| M3 (filtering)       | 7,67          | 15,21            | 12,5         | 51,9            |
| M4 (agregasi)        | 7,82          | 11,78            | 13,4         | 34,8            |

Rasio p95 GraphQL/REST per skenario: M1 ≈2,0× · M2 ≈1,7× · M3 ≈1,9× · M4 ≈1,5× (rentang 1,46–3,14); CPU 34/36 dan RSS 31/36 sel pro-REST.



# Median per sel M1–M4 pada r80 (n = 30 per sel).

| Sel       | p50 R | p50 G | p95 R | p95 G | $\delta$ |
|-----------|-------|-------|-------|-------|----------|
| M1·low    | 7,24  | 12,70 | 8,96  | 15,41 | -1,00    |
| M1·medium | 7,80  | 15,67 | 9,67  | 19,55 | -1,00    |
| M1·high   | 8,70  | 20,94 | 10,73 | 33,69 | -1,00    |
| M2·low    | 7,18  | 11,87 | 8,87  | 14,39 | -1,00    |
| M2·medium | 7,51  | 13,22 | 9,41  | 16,74 | -1,00    |
| M2·high   | 8,22  | 15,57 | 10,08 | 19,91 | -1,00    |
| M3·low    | 7,16  | 13,35 | 8,85  | 16,82 | -1,00    |
| M3·medium | 7,29  | 13,69 | 9,01  | 17,38 | -1,00    |
| M3·high   | 7,67  | 15,21 | 9,42  | 19,21 | -1,00    |
| M4·low    | 6,93  | 10,86 | 8,70  | 14,11 | -1,00    |
| M4·medium | 7,28  | 11,21 | 9,09  | 14,49 | -1,00    |
| M4·high   | 7,82  | 11,78 | 9,66  | 15,03 | -1,00    |

$\delta$  = signed Cliff's delta latency (−1,00 = pemisahan sempurna pro-REST); p (Holm) < 0,001 pada seluruh sel latensi (Tabel L.1).

## Page latency M5/M6 pada r40 (median, n = 30) + titik impas.

| Sel    | REST (ms) | GraphQL (ms) | round-trip |
|--------|-----------|--------------|------------|
| M5·w2  | 14,79     | 14,73        | 2 vs 1     |
| M5·w8  | 15,14     | 16,23        | 2 vs 1     |
| M5·w23 | 15,67     | 20,07        | 2 vs 1     |
| M6·k1  | 7,75      | 14,32        | 1 vs 1     |
| M6·k5  | 36,57     | 16,88        | 5 vs 1     |
| M6·k10 | 75,13     | 19,86        | 10 vs 1    |

K\* = 2 (interpolasi  $\Delta$  page-latency).  $\Delta$  M6·k1 = -6,58 ms (CI95 -6,90...-6,49);  $\Delta$  M5·w2 = +0,06 ms (CI95 -0,01...+0,10).

# Grid caching per blok: hit rate & Δ latency saat cache aktif (n = 30).

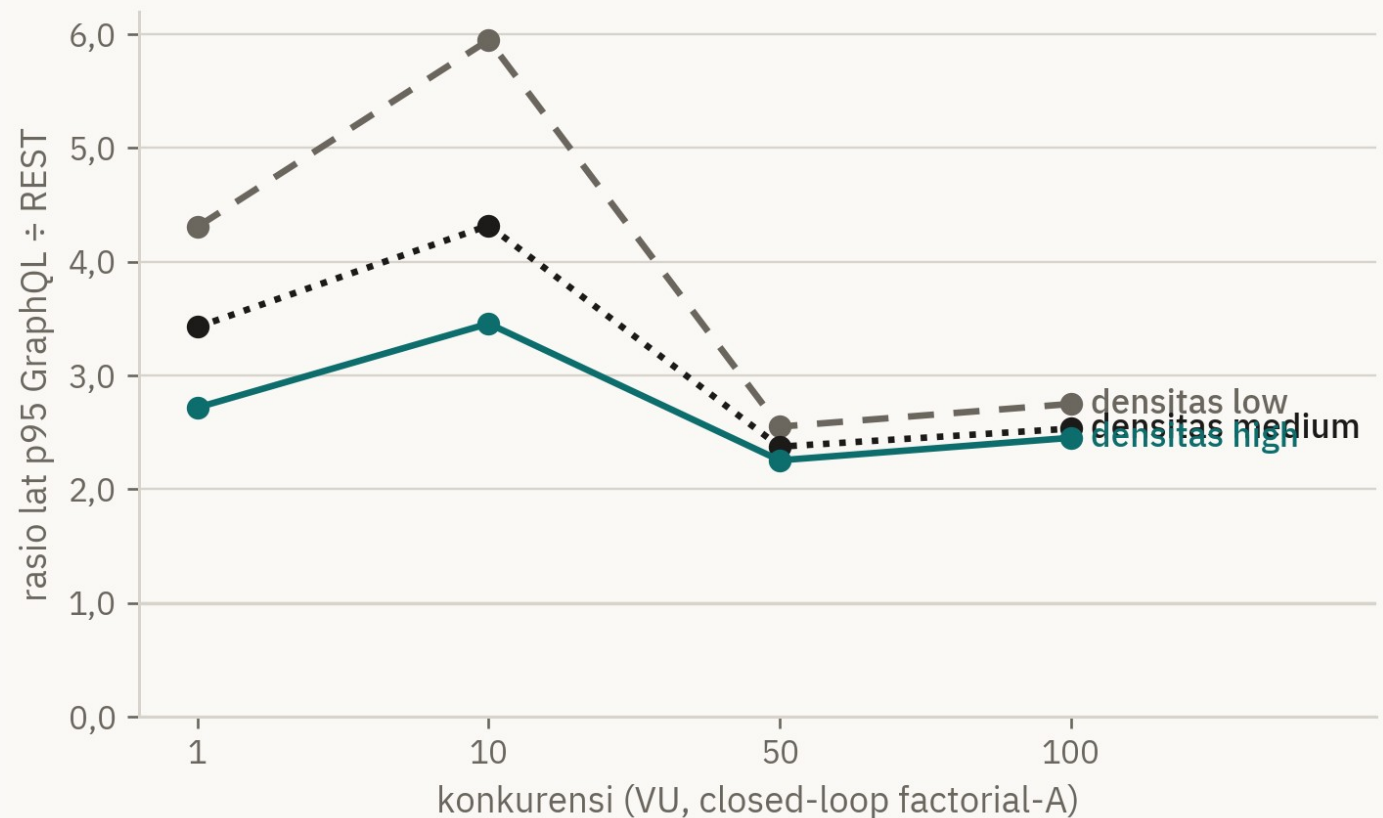
| Blok          | hit R | hit G | Δlat R (ms) | Δlat G (ms) |
|---------------|-------|-------|-------------|-------------|
| zipfian·light | 0,43  | 0,42  | -0,41       | -3,09       |
| zipfian·heavy | 0,37  | 0,37  | -0,27       | -2,21       |
| uniform·light | 0,06  | 0,06  | +0,68       | +0,87       |
| uniform·heavy | 0,02  | 0,03  | +0,99       | +0,91       |
| unique·light  | 0,00  | 0,00  | +1,01       | +0,87       |
| unique·heavy  | 0,00  | 0,00  | +0,94       | +1,07       |

# Factorial-A: arah temuan tidak pernah berbalik oleh beban — 48/48 sel pemisahan sempurna.

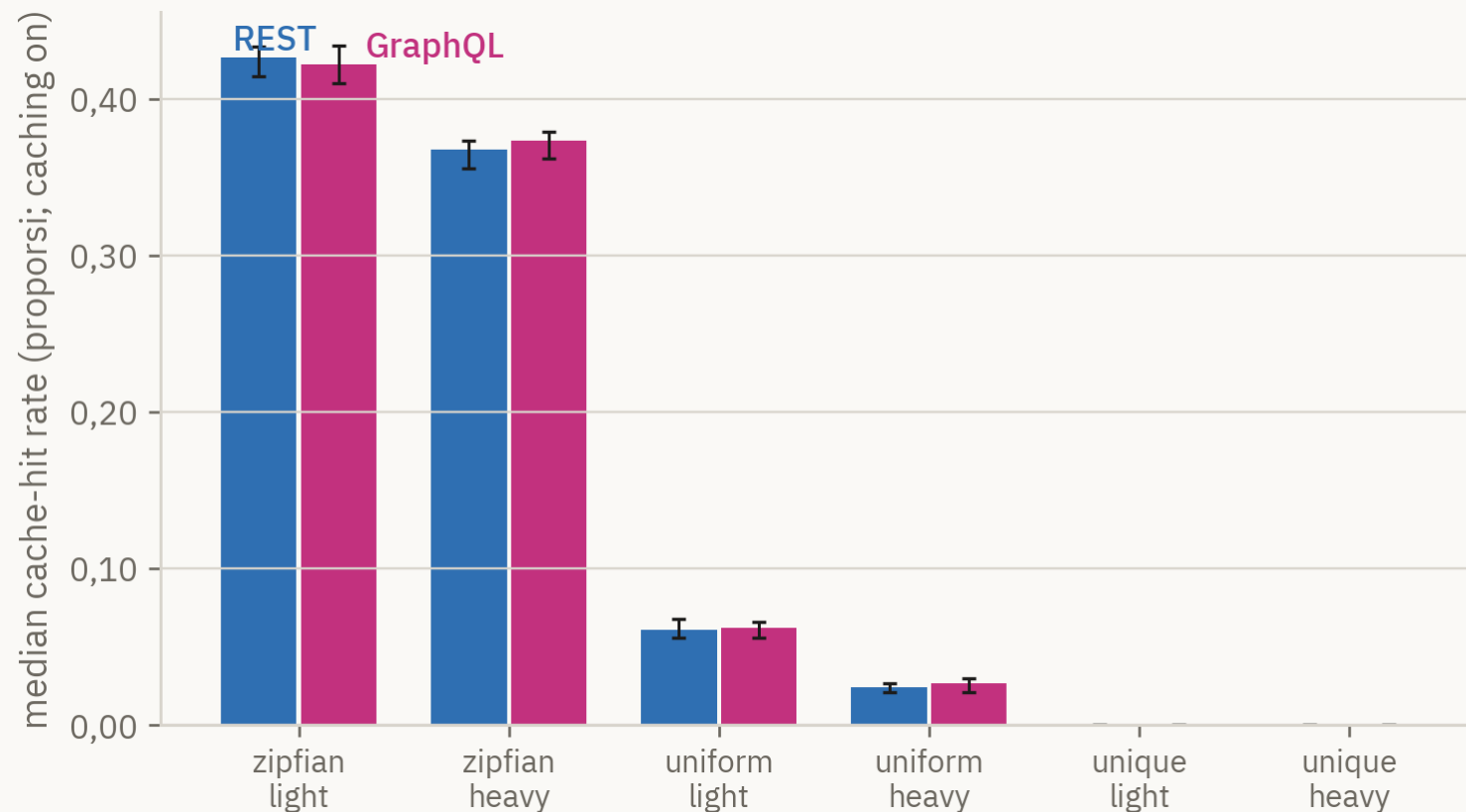
**2.880 run, 48 sel (pattern × densitas × konkurensi 1–100):**  $|\delta|$  lat\_p95 = 1,00 di seluruh 48 sel.

**188/192 perbandingan signifikan (BH-FDR):** lat\_p95 48/48, throughput 48/48, xproc\_p95 48/48 pro-REST; payload 44/48 signifikan.

**Rasio p95 per pattern:** aggregate 3,6× · filtered 2,9× · partial 2,7× · baseline 2,7× (maks 7,1×); Jonckheere-Terpstra: monoton naik terhadap konkurensi utk kedua protokol.



# Distribusi cache-hit rate: pola akses menentukan, protokol tidak.



Zipfian 0,37–0,43 · uniform 0,02–0,06 · unique 0,00; whisker = IQR 30 run; payload heavy menurunkan hit rate (entri lebih besar, variasi komposisi lebih tinggi).

# 7

## Lampiran — Metode Statistik

# Formulasi: Mann-Whitney U + rank-biserial ( $\equiv A12 \equiv \text{Cliff's } \delta$ ); keputusan bertumpu effect size.

**Uji U:** gabung-rank dua kelompok independen ( $n_1=n_2=30$ ); U yang lebih kecil dibandingkan ke distribusi U; tingkat kepercayaan 95%.

**Effect size:** rank-biserial via simple difference formula (Kerby 2014); setara A12 Vargha-Delaney dan Cliff's  $\delta$ ; tanda = arah ( $\delta < 0$  = REST unggul).

**Ambang magnitudo pipeline (Romano):**  $|\delta|$  0,147 kecil · 0,33 sedang · 0,474 besar — catatan: Bab III menulis 0,10/0,30/0,50 (Kerby), Fiel Peres 0,11/0,28/0,43; dampak praktis nol karena hampir semua  $|\delta| = 1,00$ .

# Koreksi berganda & disiplin overload: Holm per keluarga metrik; r120 tidak pernah dipool.

**Eksperimen utama:** koreksi Holm diterapkan per keluarga metrik (implementasi `analyze_phase2.py` / `analyze_mot_scenarios.py`).

**Uji pendahuluan:** Benjamini-Hochberg FDR  $q = 0,05$  — hanya di analisis pilot.

**Overload:** r120\_overload dianalisis dalam keluarga Holm terpisah per family skenario; kolom `overload_saturates` mencatat protokol pendefinisi ceiling; jangan bandingkan overload lintas keluarga.

**Catatan konsistensi:** satu kalimat Ancaman Validitas Bab V masih menulis 'Benjamini-Hochberg' untuk grid inti — koreksi F5: Holm (BH hanya pilot).



# 8

## Lampiran — Insiden & Integritas Data

# Taksonomi error tersembunyi run-sesi-1 (uji pendahuluan).

| Aspek                 | Nilai                                                                                                           |
|-----------------------|-----------------------------------------------------------------------------------------------------------------|
| Jenis error           | ValueError: could not convert string to float: 'x1'<br>+ GraphQLError: Float cannot represent non numeric value |
| Lokasi                | serialisasi field bounding_box (GraphQL)                                                                        |
| Jumlah traceback      | 345.879 (berpasangan)                                                                                           |
| Rentang di log        | ~0,1% – 59,5% offset log 1,34 GB, lalu berhenti                                                                 |
| Kenapa tersembunyi    | HTTP 200 in-band; cek k6 hanya status===200                                                                     |
| error_rate di CSV     | 0 di semua baris (tidak terlihat)                                                                               |
| Iterasi GraphQL total | 7.947.457 (baseline+filtered: 3.297.716)                                                                        |
| Sesi lain             | run-sesi-2: 0 error · factorial-A: 0 error                                                                      |
| Konteks tambahan      | run-sesi-1/2 pra-perbaiki fairness 824b195                                                                      |

Jumlah request terdampak tidak dapat diturunkan dari log (satu error per kemunculan field).

# Konsekuensi & mitigasi: DEMOTED – dan guard permanen agar kelas error ini tidak bisa lolos lagi.

**Vonis (2026-07-09):** run-sesi-1 dikeluarkan dari SEMUA angka tesis; pengganti Study-A: factorial-A (2.880 run, audit bersih 0 error).

**Guard permanen:** k6 body-check “no graphql errors” pada respons final (aman terhadap retry registrasi APQ).

**Kontrol negatif diverifikasi:** kueri field invalid = HTTP 200 + body errors → tertangkap check; smoke test 9/9 blok PASS; APQ registrasi tidak menjatuhkan check (51 pass/0 fail).

## Timeline kontaminasi konkurensi phase2 → definisi subset bersih.

| Sesi                        | Run         | Sesi paralel saat berjalan       |
|-----------------------------|-------------|----------------------------------|
| phase2-core-real            | 720         | 1 (SERIAL PENUH – subset bersih) |
| phase2-density-drillin      | 720         | 1-8 (332 run pada 8)             |
| phase2-entropy-drillin      | 1.440       | 1-8 bervariasi                   |
| phase2-batch-real           | 1.080       | 2-8                              |
| phase2-core-cpu-rerun       | 720         | 4-8 (tidak pernah serial)        |
| phase2-concurrency-drillin  | 1.440       | 1-8                              |
| phase2-network-drillin      | 720         | 5-8                              |
| network×conc / entropy×conc | 720 / 1.440 | 4-8 / 1-8                        |
| phase2-concurrency100       | 720         | mayoritas 1; 234 run pada 3      |
| mot-scenarios-core          | 3.240       | 1 (serial penuh)                 |

9.000 dari 9.720 baris phase2 (92,6%) terkumpul saat 2-8 sesi berjalan → RQ3 memakai HANYA phase2-core-real; drill-in = korroborasi paling banter.

# Bug telemetry CPU/RSS: ditemukan → diperbaiki → direplikasi penuh pada re-run bersih.

**Cacat:** sampler memantau PID pembungkus (sudo), bukan proses server → kolom CPU/RSS phase2-core-real invalid; metrik lain (k6-path: latency, payload, hit rate) tidak terdampak.

**Perbaiki:** sebelum studi MOT dijalankan; seluruh klaim CPU/memori Bab V bersumber dari korpus MOT.

**Re-run phase2-core-clean (720 run, desain identik):** mereplikasi 12/12 blok —  $\delta$  latensi tetap  $-1,00$ , pergeseran median  $\leq 0,25$  ms, hit rate identik hingga 3 desimal → CPU/RSS valid dilaporkan di Tabel L.2; konsisten dengan run verifikasi independen kedua (phase2-core-cpu-rerun).

# Audit integritas mot-scenarios-core: lulus setiap kriteria — termasuk dua disclosure kecil.

**Kelengkapan:** 3.240/3.240 baris (md5 94866487...); 108 sel × tepat 30 run; error\_rate 0 di semua baris.

**In-band audit:** 3.240/3.240 k6 summaries terparse, 0 failed checks; log server 2,8 GB: 0

Traceback/ValueError/GraphQLError.

**Overload by design:** dropped\_iterations>0 hanya di 91 baris r120 (M1-high·gql, M5-w23·gql, M6-k10·rest, M4-high·rest×1).

**Disclosure:** identitas DB dibuktikan hash dump logis (cdb7c9f4...) karena file berpindah lokasi pasca-run; cpu\_governor.txt kosong (VM tanpa cpufreq).

# 9

## Lampiran — Fairness & Protokol

# Acceptance tests A1–A3: paritas dan fairness diuji otomatis, dengan bukti eksekusi nyata.

**A1 paritas data:** respons deep-equal REST vs GraphQL pada 3 tier densitas, kondisi filter (class\_id, min\_confidence), dan kasus 404 (tests/test\_parity.py — seluruh assertion lolos).

**A2 jalur akses:** APE\_LOG\_SQL=1 → SQL kedua protokol di-diff tabel & predikatnya — bukan diasumsikan sama.

**A3 fairness cache:** Varnish nyata via subprocess: MISS→HIT dengan ETag identik di kedua protokol; no-store dihormati; hash APQ mismatch ditolak (PERSISTED\_QUERY\_HASH\_MISMATCH).

**Gate MOT:** 18 passed / 2 skipped (skip = asersi SQL-log khusus sqlite pada backend memory); delta envelope konstan 19 B (M1–M3) / 13 B (M4); drift guard: query test suite  $\equiv$  query k6.



# Protokol wire APQ-over-GET: hash dulu, registrasi sekali, reuse selamanya — dan biayanya terukur.

**Langkah 1:** GET /graphql?extensions={persistedQuery:{sha256Hash}}&variables=... (hash-only, cacheable).

**Langkah 2 (miss):** server balas PERSISTED\_QUERY\_NOT\_FOUND → klien kirim ulang + teks query; server verifikasi sha256(query) == hash lalu simpan.

**Terukur, bukan diasumsikan:** kolom apq\_registrations merekam biaya round-trip registrasi.

**Batas:** store proses-lokal (single worker uvicorn); multi-worker butuh store bersama — di luar lingkup.

# DataLoader batching: GraphQL diuji dalam bentuk kompetennya — bukan strawman N+1.

**Resolusi `Detection.track()`:** strawberry.dataloader membatch ke SATU query SQL berklausula IN per batch — bukan satu query per deteksi.

**Arm N+1 sengaja:** APE\_GRAPHQL\_BATCHING=off tersedia untuk studi sisipan yang mengkuantifikasi penalti N+1 — bukan perbandingan utama.

**Status:** belum dikuantifikasi terpisah → butir Saran Bab VII (WunderGraph 2026: batching dapat menurunkan cache-hit rate meski mengurangi round-trip).

# 10

## Lampiran — Q&A & Referensi

# Lima serangan penguji yang paling mungkin – dan amunisinya.

**“Lawi 2021: GraphQL lebih hemat CPU – Anda 4x?”** Beda titik ukur (gateway sistem utuh vs cgroup+psutil proses server), beda load model (closed-loop JMeter vs open-loop laju identik), beda stack; konvergen Elghazal 2025; direplikasi 2 run independen (core-clean, cpu-rerun).

**“ $\delta = \pm 1,00$  di mana-mana – mencurigakan?”** Gap konsisten + varians kecil ( $n=30$ , lingkungan terkontrol) =  $\delta$  jenuh; justru bukti kontrol; selisih praktis dilaporkan terpisah (ms/%) — lihat non-temuan throughput.

**“REST Anda tailored; GraphQL tak diberi kesempatan?”** By design: yang diukur overhead protokol pada fungsionalitas setara; keunggulan konsolidasi GraphQL tetap terukur dan menang di M6  $K \geq 5$ ; preseden dua ujung: Brito 2019 (REST generik, -99%).

**“Generalisasi?”** Ancaman validitas eksplisit (1 mesin, 1 stack, netem, 7 sequence); prediksi arah: RTT besar menggeser crossover pro-GraphQL (Jin et al. 2024).

# Peta vonis hipotesis — H1 didukung; H2 sebagian; H3 mekanistik; H4 menunggu kalimat vonis di tesis.

**H1 — DIDUKUNG:** REST latency lebih rendah di seluruh sel M1–M4 tanpa cache; pemisahan sempurna (Bab VII ¶RQ1).

**H2 — DIDUKUNG SEBAGIAN:** APQ menyetarakan cacheability & hit rate; caching mempersempit tapi tidak menghapus selisih; manfaat bersyarat pola akses berulang (Bab VII ¶RQ3).

**H3 — KONSISTEN MEKANISTIS:** faktor entropi dikunci medium di grid inti; prediksi didukung sifat cache key sebagai fungsi teks kueri. Catatan: drill-in entropi 1.440 run ada di results/ namun terdampak paralelisme → korroborasi saja.

**H4 — [VERIFY: vonis H4 belum eksplisit di Bab V/VII]:** data pendukung & pembantah tersedia (densitas M1–M4, window M5 vs label payload grid caching); kalimat vonis diusulkan terpisah — slide di-update setelah teks tesis final.

# Daftar pustaka (pasca-audit) – 1–16 dari 45.

## 1–8

---

Ala-Laurinaho, R., Mattila, J., Autiosalo, J., Hietala, J., Laaki, H., & Tammi, K. (2022). Comparison of REST and GraphQL Interfaces for OPC UA. *Computers (MDPI)*.  
<https://doi.org/10.3390/computers11050065>

Ali, M. L. & Zhang, Z. (2024). The YOLO Framework: A Comprehensive Review of Evolution, Applications, and Benchmarks in Object Detection. *Computers (MDPI)* 13(12):336.  
<https://doi.org/10.3390/computers13120336>

Alonso, G., Casati, F., Kuno, H., & Machiraju, V. (2004). *Web Services: Concepts, Architectures and Applications*. Springer. <https://doi.org/10.1007/978-3-662-10876-5>

Apollo GraphQL (2026). Automatic Persisted Queries (APQ) - Dokumentasi. Apollo Docs.  
<https://www.apollographql.com/docs/apollo-server/performance/apq>

Arcuri, A. & Briand, L. (2011). A Practical Guide for Using Statistical Tests to Assess Randomized Algorithms in Software Engineering. *ICSE 2011*. <https://doi.org/10.1145/1985793.1985795>

Bang, J., Kakkar, G. T., Chunduri, P., Mitra, S., & Arulraj, J. (2023). Seiden: Revisiting Query Processing in Video Database Systems. *PVLDB* 16(9).  
<https://doi.org/10.14778/3598581.3598599>

Breslau, L., Cao, P., Fan, L., Phillips, G., & Shenker, S. (1999). Web Caching and Zipf-like Distributions: Evidence and Implications. *IEEE INFOCOM 1999*.  
<https://doi.org/10.1109/INFCOM.1999.749260>

Brito, G. & Valente, M. T. (2020). REST vs GraphQL: A Controlled Experiment. *IEEE ICSA 2020*.  
<https://arxiv.org/abs/2003.04761>

## 9–16

---

Brito, G., Mombach, T., & Valente, M. T. (2019). Migrating to GraphQL: A Practical Assessment. *IEEE SANER 2019*. <https://arxiv.org/abs/1906.07535>

Cha, A., Wittern, E., Baudart, G., Davis, J. C., Mandel, L., & Laredo, J. A. (2020). A Principled Approach to GraphQL Query Cost Analysis. *ESEC/FSE 2020 (ACM)*.  
<https://doi.org/10.1145/3368089.3409670>

Crankshaw, D., Wang, X., Zhou, G., Franklin, M. J., Gonzalez, J. E., & Stoica, I. (2017). Clipper: A Low-Latency Online Prediction Serving System. *NSDI 2017*.  
<https://www.usenix.org/conference/nsdi17/technical-sessions/presentation/crankshaw>

De Rosa, P., Bromberg, Y.-D., Felber, P., Mvondo, D., & Schiavoni, V. (2024). On the Cost of Model-Serving Frameworks: An Experimental Evaluation. *IEEE IC2E 2024*, pp. 221-232.  
<https://doi.org/10.1109/IC2E61754.2024.00032>

Elghazal, M. S. M., Aneiba, A., & Shahra, E. Q. (2025). Performance Evaluation of REST and GraphQL API Models in Microservices Software Development Domain. *WEBIST 2025 (Scitepress)*, pp. 83-91. <https://www.scitepress.org/Papers/2025/137299/137299.pdf>

Fiel Peres, F. (2025). Effect sizes for nonparametric tests (review). *Biochemia Medica*.  
<https://doi.org/10.11613/BM.2026.010101>

Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures*. UC Irvine (Disertasi PhD).  
[https://roy.gbiv.com/pubs/dissertation/fielding\\_dissertation.pdf](https://roy.gbiv.com/pubs/dissertation/fielding_dissertation.pdf)

Fielding, R. T. & Taylor, R. N. (2002). Principled Design of the Modern Web Architecture. *ACM Transactions on Internet Technology (TOIT)*. <https://doi.org/10.1145/514183.514185>

# Daftar pustaka (pasca-audit) – 17–32 dari 45.

## 17–24

---

Fielding, R., Nottingham, M., & Reschke, J. (2022). RFC 9111: HTTP Caching. IETF (Internet Standard). <https://doi.org/10.17487/RFC9111>

Goriparthi, S. (2024). Enhancing Data Query Efficiency with GraphQL APIs: A Comparative Study with RESTful Web Services. *International Journal of Artificial Intelligence & Machine Learning (IJAIML)* Vol 3(1), 2024, p.132-143 -- IAEME Publication. [https://iaeme.com/Home/article\\_id/IJAIML\\_03\\_01\\_013](https://iaeme.com/Home/article_id/IJAIML_03_01_013)

GraphQL Foundation (2026). Performance (Persisted Queries & Caching). [graphql.org/learn/](https://graphql.org/learn/). <https://graphql.org/learn/performance/>

Hartig, O. & Perez, J. (2018). Semantics and Complexity of GraphQL. *The Web Conference (WWW 2018)*. <https://doi.org/10.1145/3178876.3186014>

Jackson, S., Cummings, N., & Khan, S. (2024). Streaming Technologies and Serialization Protocols: Empirical Performance Analysis. *IEEE Access* 12. <https://doi.org/10.1109/ACCESS.2024.3486054>

Jin, R., Cordingly, R., Zhao, D., & Lloyd, W. (2024). GraphQL vs. REST: A Performance and Cost Investigation for Serverless Applications. *WOSC 2024 (ACM)*. <https://doi.org/10.1145/3702634.3702956>

Jin, R., Cordingly, R., Zhao, D., & Lloyd, W. (2025). GraphQL vs. REST: Investigating Performance and Scalability for Serverless Data Persistence. *IEEE IC2E 2025*. <https://faculty.washington.edu/wlloyd/papers/graphql-camera-ready.pdf>

Kang, D., Bailis, P., & Zaharia, M. (2020). BlazeIt: Optimizing Declarative Aggregation and Limit Queries for Neural Network-Based Video Analytics. *PVLDB* 13(4). <https://arxiv.org/abs/1805.01046>

## 25–32

---

Kerby, D. S. (2014). The Simple Difference Formula: An Approach to Teaching Nonparametric Correlation. *Comprehensive Psychology*, 3. <https://doi.org/10.2466/11.IT.3.1>

Lawi, A., Panggabean, B. L. E., & Yoshida, T. (2021). Evaluating GraphQL and REST API Services Performance in a Massive and Intensive Accessible Information System. *Computers (MDPI)*. <https://doi.org/10.3390/computers10110138>

Mann, H. B. & Whitney, D. R. (1947). On a Test of Whether one of Two Random Variables is Stochastically Larger than the Other. *The Annals of Mathematical Statistics*, 18(1). <https://doi.org/10.1214/aoms/1177730491>

Masse, M. (2011). *REST API Design Rulebook*. O'Reilly Media. <https://www.oreilly.com/library/view/rest-api-design/9781449317904/>

Nieto, M., Otaegui, O., & Senderos, O. (2021). Boosting AI applications: Labeling format for complex datasets. *SoftwareX*. <https://doi.org/10.1016/j.softx.2020.100653>

Niswar, M., Safruddin, R. A., Bustamin, A., & Aswad, I. (2024). Performance Evaluation of Microservices Communication with REST, GraphQL, and gRPC. *Int. Journal of Electronics and Telecommunications*. <https://doi.org/10.24425/ijet.2024.149562>

Pantelic, N., Matic, L., Jakovljevic, L., Eric, S., Eric, M., Stefanovic, M., & Djordjevic, A. (2026). Benchmarking SQL and NoSQL Persistence in Microservices Under Variable Workloads. *Future Internet (MDPI)* 18(1):53. <https://doi.org/10.3390/fi18010053>

Pressman, R. S., & Maxim, B. R. (2014). *Software Engineering: A Practitioner's Approach* (8th ed.). McGraw-Hill Education.

# Daftar pustaka (pasca-audit) — 33–45 dari 45.

## 33–39

---

Qiu, Z., Yang, J., & Harchol-Balter, M. (2024). Can Increasing the Hit Ratio Hurt Cache Throughput?. ACM ValueTools 2024 / arXiv:2404.16219. <https://arxiv.org/abs/2404.16219>

Quina-Mera, A., Fernandez, P., Garcia, J. M., & Ruiz-Cortes, A. (2023). GraphQL: A Systematic Mapping Study. ACM Computing Surveys. <https://doi.org/10.1145/3561818>

Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You Only Look Once: Unified, Real-Time Object Detection. CVPR 2016. <https://doi.org/10.1109/CVPR.2016.91>

Sapkota, R. & Karkee, M. (2025). Ultralytics YOLO Evolution: An Overview of YOLO26, YOLO11, YOLOv8 and YOLOv5 Object Detectors for Computer Vision and Pattern Recognition. arXiv:2510.09653. <https://arxiv.org/abs/2510.09653>

Seabra, M., Nazário, M. F., & Pinto, G. (2019). REST or GraphQL? A Performance Comparative Study. SBCARS 2019 (ACM). <https://doi.org/10.1145/3357141.3357149>

Stepien, K. & Skublewska-Paszkowska, M. (2025). Performance evaluation of REST and GraphQL API approaches in data retrieval scenarios using NestJS. Journal of Computer Sciences Institute. <https://doi.org/10.35784/jcsi.7794>

Tene, G. (2015). How NOT to Measure Latency [Conference presentation]. Strange Loop 2015.

## 40–45

---

Tomczak, M. & Tomczak, E. (2014). The Need to Report Effect Size Estimates Revisited. An Overview of Some Recommended Measures of Effect Size. Trends in Sport Sciences, 1. [https://www.wbc.poznan.pl/Content/325867/5\\_Trends\\_Vol21\\_2014\\_%20no1\\_20.pdf](https://www.wbc.poznan.pl/Content/325867/5_Trends_Vol21_2014_%20no1_20.pdf)

Vargha, A. & Delaney, H. D. (2000). A Critique and Improvement of the CL Common Language Effect Size Statistics of McGraw and Wong. Journal of Educational and Behavioral Statistics, 25(2). <https://doi.org/10.3102/10769986025002101>

Wittern, E., Cha, A., Davis, J. C., Baudart, G., & Mandel, L. (2019). An Empirical Study of GraphQL Schemas. ICSOC 2019 (Springer LNCS). [https://doi.org/10.1007/978-3-030-33702-5\\_1](https://doi.org/10.1007/978-3-030-33702-5_1)

Wohlin, C., Runeson, P., Host, M., Ohlsson, M. C., Regnell, B., & Wesslen, A. (2024). Experimentation in Software Engineering (2nd ed.). Springer. <https://doi.org/10.1007/978-3-662-69306-3>

WunderGraph (2026). GraphQL vs REST: Claims Fact-Checked with Primary Sources. WunderGraph (blog teknis). <https://wundergraph.com/blog/fact-checking-graphql-vs-rest>

Zhu, P., Wen, L., Du, D., Bian, X., Fan, H., Hu, Q., & Ling, H. (2021). Detection and Tracking Meet Drones Challenge (VisDrone). IEEE TPAMI 44(11):7380-7399. <https://doi.org/10.1109/TPAMI.2021.3119563>